Database Scaling

What is Database Scaling?

Database scaling means **increasing a database's ability to handle more data, more users, and more queries without slowing down or crashing.**

In simple terms:

Scaling = making the database handle higher load.

Load can mean:

- more **users**
- more **read queries**
- more **write queries**
- more **stored data**

Database scaling is the process of increasing a database system's capacity to handle more traffic, data, and queries by upgrading hardware (vertical scaling) or distributing load across multiple servers (horizontal scaling).

Why Database Scaling Is Needed.?

Short answer:

A single database server has **limited resources**, but user traffic can grow **infinitely**.

Those limits create **bottlenecks**.

1️⃣ Limited CPU

Every query needs CPU.

Example:

```
SELECT * FROM users WHERE email = 'x'
```

If:

- 10 users → easy
- 10,000 users → still fine
- 1,000,000 users → CPU overload 🔥

The database simply **cannot process queries fast enough**.

2️⃣ Limited Memory (RAM)

Databases keep frequently accessed data in memory.

Example:

- DB RAM = **16 GB**
- Data size = **2 TB**

Now most queries must fetch from **disk instead of RAM**, which is **much slower**.

Result:

- slow queries
- slow API responses

3️⃣ Disk I/O Limits

Reading from disk is expensive.

Example:

Full table scan

If a table has **50 million rows**, the disk must scan everything.

Disk becomes the **biggest bottleneck**.

4️⃣ Connection Limits

Databases cannot handle unlimited connections.

Example for **PostgreSQL**:

Typical safe range:

100–500 active connections

But imagine:

10,000 users hitting your API

DB gets overwhelmed.

5️⃣ Storage Limits

Eventually data becomes huge.

Example:

Users table → 500M rows
Orders table → 2B rows
Logs → 10TB

Single server storage becomes insufficient.

6️⃣ Single Point of Failure

If you have **only one database**:

App → DB

And the DB crashes 💥

Everything goes down.

Your entire app is offline.

This is unacceptable for large systems.

Real Example

Think of a company like Instagram.

Millions of users:

- posting photos
- liking posts
- opening feeds

- commenting

Each action = **database query**.

Without scaling, their DB would collapse.

The Real Root Problem

Database scaling solves **4 core limits**:

1️⃣ **CPU limits**
2️⃣ **Memory limits**
3️⃣ **Disk I/O limits**
4️⃣ **Connection limits**

When traffic grows, these limits break the system.

Database Bottlenecks

A **database bottleneck** is **the part of the database system that limits overall performance** when traffic increases.

In simple terms:

A bottleneck is the **weakest component that slows down the entire system**.

Like a traffic jam 🚗 — even if the highway is wide, a **single narrow lane** slows everyone down.

1️⃣ Connection Bottleneck

Too many clients trying to connect.

10,000 users → API → Database

But the database allows only **300 connections**.

Result:

ERROR: too many connections

This commonly happens in databases like **PostgreSQL** and **MySQL**.

Solution usually:

- connection pooling
- pooling proxies like **PgBouncer**

2️⃣ Query Bottleneck

Some queries are **very slow**.

Example:

SELECT * FROM users WHERE email='abc@gmail.com'

If **no index exists**, the database must scan the entire table.

10 million rows scanned

Result:

- CPU usage increases
- queries take seconds

Solution:

- indexing
- query optimization

3️⃣ Disk I/O Bottleneck

Databases constantly **read and write data to disk**.

If the disk cannot keep up:

Queries wait for disk

Symptoms:

- high latency
- slow writes
- slow reads

Common when:

- tables are huge
- indexes missing
- too many writes

4️⃣ CPU Bottleneck

Heavy queries can overload CPU.

Examples:

- large joins
- aggregations
- sorting

Example:

```
SELECT COUNT(*) FROM logs;
```

If logs table has **500M rows**, CPU works very hard.

5️⃣ Memory Bottleneck

Databases rely heavily on **RAM caching**.

If RAM is small:

Data must be fetched from disk

Disk access is **100× slower than RAM**.

Result:

- slow queries
- slow responses

The Key Insight (very important)

When a database slows down, **only one or two resources are usually saturated**:

CPU
Memory
Disk I/O
Connections

Good engineers **identify the bottleneck first** before scaling.

Golden Rule of Scaling

Senior engineers follow this rule:

Never scale blindly. First identify the bottleneck.

Otherwise you might:

- add servers
- increase RAM
- shard databases

…but the real issue was just **a missing index**.

Query Optimization

Query optimization means **writing or structuring database queries so they run as fast and efficiently as possible.**

Query optimization is the process of improving database queries so they retrieve data efficiently by minimizing resource usage like CPU, memory, and disk I/O.

Goal:

Return the required data using **minimum CPU, memory, and disk I/O**.

Databases like **PostgreSQL** and **MySQL** internally try to optimize queries, but **bad queries still happen a lot**.

Why Query Optimization Matters

Imagine a table:

users
---

id
name
email
created_at

Now a query:

```
SELECT * FROM users WHERE email = 'abc@gmail.com';
```

If the table has:

50 million rows

and **no index**, the database must **scan every row**.

That is called a **full table scan**.

Result:

slow query
high CPU
high disk usage

Common Query Optimization Techniques

1 Indexing (Most Important)

Create an index on frequently searched columns.

2 Avoid SELECT *

Bad practice:

```
SELECT * FROM users;
```

This loads **all columns**, even if you only need one.

Better:

```
SELECT name FROM users;
```

Less data → faster query.

3 Use LIMIT

If you don’t need all results.

Bad:

```
SELECT * FROM orders;
```

Better:

```
SELECT * FROM orders LIMIT 20;
```

This is common in **pagination**.

4️⃣ Avoid Unnecessary Joins

Bad query example:

```
SELECT *
FROM users
JOIN orders
JOIN payments
JOIN products
```

Multiple joins increase computation cost.

Better approach:

- fetch only needed tables
- split queries if required

5️⃣ Use Query Plans

Databases show **how they execute queries**.

Example in **PostgreSQL**:

```
EXPLAIN ANALYZE
SELECT * FROM users WHERE email='abc@gmail.com';
```

It tells you:

- whether index is used
- how many rows scanned
- execution time

This is how engineers debug slow queries.

Signs You Have a Bad Query

Typical symptoms:

- API endpoint suddenly slow
- CPU spikes on DB
- high disk I/O
- slow query logs

Most of the time the issue is **a bad query or missing index**.

Real Example

Imagine a social feed like **Instagram**.

Query for feed:

```
SELECT *
FROM posts
WHERE user_id IN (followers list)
ORDER BY created_at DESC
LIMIT 20;
```

If this query isn't optimized:

- millions of rows scanned
- feed loads slowly

With proper indexing:

(user_id, created_at)

Feed loads **instantly**.

Indexing — B-Tree Indexes

Definition:

An index is a **data structure that allows the database to find rows quickly without scanning the entire table**.

Think of it like the **index at the back of a book**. 📚 Instead of reading every page, you jump directly to the page you need.

1️⃣ Why Indexes Are Needed

Without an index:

SELECT * FROM users WHERE email='abc@gmail.com';

- Table has **50M rows**
- Database does a **full table scan**
- Query is **slow**, CPU-heavy, disk-heavy

With an index:

- Database can **jump to the relevant row** directly
- Query is **much faster** (milliseconds vs seconds)

2️⃣ B-Tree Indexes — How They Work

Think of a B-Tree index like a telephone directory:

- **You want Alice Smith → check the index → go to the exact page**
- **Instead of scanning every page one by one**

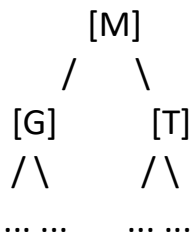
Even with millions of contacts, it's extremely fast because the tree branches efficiently.

B-Tree = **Balanced Tree**

- Every node has multiple keys

- Tree is **balanced**, meaning all leaf nodes are at the same level
- Lookup, insert, delete = **O(log n)**

  Visual:



- Root → Middle node → Leaf node → Actual row
- Very fast even for **millions of rows**

Key Properties

1. **Sorted** → good for range queries:

```
SELECT * FROM users WHERE id BETWEEN 100 AND 200;
```

2. **Balanced** → same lookup time no matter which value you search
3. **Supports equality and range queries** (= , <, >, BETWEEN)

B-Tree indexes are great for:

- Exact matches (=)
- Range queries (>, <, BETWEEN)
- Ordered queries (ORDER BY)

Not ideal for:

- Full-text search → use **GIN or inverted indexes**
- High-frequency writes → each write must update the index (small overhead)
- Boolean or low-cardinality columns → index may not help

3️⃣ Example — Creating Index in PostgreSQL

```
CREATE INDEX idx_email ON users(email);
```

Now queries like this are fast:

```
SELECT name FROM users WHERE email='abc@gmail.com';
```

4️⃣ Multi-Column (Composite) Index

Sometimes queries use **multiple columns**:

```
SELECT * FROM orders
WHERE user_id=123 AND status='shipped';
```

You can create a composite index:

```
CREATE INDEX idx_user_status ON orders(user_id, status);
```

Now the database can **efficiently use the index** instead of scanning the whole table.

Indexes are **the first step in scaling** because:

- Queries run faster → less CPU → less memory → less disk I/O
- Reduces need for **read replicas** or caching initially
- Essential before you think about **sharding**

In short:

Proper indexing can **delay the need to horizontally scale**, saving huge infrastructure costs. 💰

Indexing: Read vs Write Tradeoff

1️⃣ What Happens When You Add an Index

Every time you **insert, update, or delete a row**, the database must:

1. Update the **table** itself
2. Update **all indexes** on that table

So **writes become slower** as you add more indexes.

2️⃣ Quantifying the Tradeoff

Action	Without Index	With 1 Index	With 5 Indexes
SELECT	Slow (full scan)	Fast	Fast
INSERT/UPDATE	Fast	Slightly slower	Much slower
Disk I/O	Less	More	Much more

- **Reads → faster**
- **Writes → slower**

3️⃣ Practical Rule

Index **only the columns you query frequently**.

- Don't index columns you rarely search or filter on.
- Avoid over-indexing just "in case" — it kills write performance.

4️⃣ Read-Heavy vs Write-Heavy Systems

Read-Heavy Systems

- e.g., social feed, analytics dashboard
- More indexes → worth it, because queries dominate traffic

Write-Heavy Systems

- e.g., logs, messaging, payments
- Fewer indexes → writes dominate → keep inserts fast

Connection Pooling

Definition:

Connection pooling is a technique where the application reuses a fixed number of database connections instead of opening a new connection for every request.

1️⃣ Why It's Needed

Databases like PostgreSQL or MySQL can handle a limited number of concurrent connections:

- **PostgreSQL: 100–500 typical**
- **MySQL: 1000 typical**

If your app suddenly has thousands of users opening new connections for every request, the DB hits the max connections and crashes:

2️⃣ How Pooling Works

Instead of opening a new connection for every request:

Users → App → DB (new connection each request)

You maintain a pool of pre-established connections:

Users → App → Connection Pool → DB

- **Pool size = fixed (e.g., 20–100)**
- **Requests reuse existing connections**
- **Excess requests wait in queue until a connection is free**

3️⃣ Example

Without Pooling

```
// Node.js
const client = await pool.connect(); // creates new DB connection
```

```
const result = await client.query("SELECT * FROM users");
client.release();
```

- Every request may create a new connection → overhead
- Slow, high CPU, potential DB crash under load

With Pooling

```
const pool = new Pool({ max: 20 }); // max 20 connections

const result = await pool.query("SELECT * FROM users");
```

- Reuses connections → faster
- Limits DB connections → prevents overload

4️⃣ Key Benefits

1. **Reduces Connection Overhead** – creating a DB connection is expensive
2. **Limits Maximum DB Connections** – protects database from crashing
3. **Improves Throughput** – faster response under high load

5️⃣ Typical Pool Settings

Setting	Purpose
max	Maximum number of connections in pool
min	Minimum number of idle connections
idleTimeout	How long idle connections are kept
acquireTimeout	How long to wait for a free connection

6️⃣ Real-World Analogy

Imagine a restaurant:

- DB connections = **tables**

- App requests = **customers**
- No pool → every customer builds a table from scratch → chaos
- Pool → fixed tables, customers wait if busy → smooth operation

7️⃣ Relation to Scaling

Connection pooling is **often the first thing that breaks in high traffic systems**:

- You can have **optimized queries** and **powerful hardware**, but if **connections exceed DB limits**, the system fails.
- Pooling is a **cheap, simple way to scale apps immediately**.

Question: *Why do we need connection pooling?*

Answer:

Because creating new DB connections for every request is expensive and can exceed the database's connection limit under high load. Pooling reuses connections efficiently and prevents crashes.

Vertical Scaling (Scaling Up)

Real-World Analogy

Think of a kitchen:

- **Scaling vertically = getting a bigger oven, more stoves, and a bigger fridge**
- **You can cook faster, but you still have one chef — if the chef falls sick, the kitchen stops**

Definition:

Vertical scaling is the process of **increasing the resources of a single database server** to handle more load.

In other words:

- Bigger CPU
- More RAM
- Faster or larger disk

Instead of adding more servers, you **make the existing one stronger**.

1️⃣ Why Vertical Scaling Works

When a database is slow, common bottlenecks are:

- CPU maxed out
- RAM insufficient for caching
- Disk I/O saturated

Vertical scaling fixes these by giving **more resources**:

Before: 4 CPU, 16 GB RAM
After: 16 CPU, 64 GB RAM

- More CPU → faster query execution

- More RAM → larger caches → less disk access
- Faster disks → quicker reads/writes

2️⃣ Example

Imagine a Node.js app with **PostgreSQL**:

- Current DB: 4 CPU cores, 16 GB RAM
- Handles 10k requests/minute comfortably
- Now requests spike to 100k/minute → DB slows

Solution (Vertical Scaling):

- Upgrade to 16 CPU cores
- Increase RAM to 64 GB
- Use SSD/NVMe for disk

Result: DB handles the traffic **without changing application logic**.

3️⃣ Pros of Vertical Scaling

✅ Simple to implement
✅ No changes to application or database design
✅ Works immediately for CPU/memory/disk bottlenecks

4️⃣ Cons of Vertical Scaling

❌ Limited by hardware → can't scale infinitely
❌ Expensive at high-end servers
❌ Single point of failure → if the server crashes, entire DB is down

When to Use

- Small to medium apps
- First step before horizontal scaling
- When bottleneck is clearly **CPU, RAM, or disk**

Horizontal Scaling (Scaling Out)

Real-World Analogy

Think of a restaurant again 🍴:

- Vertical = bigger kitchen
- Horizontal = multiple kitchens in parallel, each cooking part of the orders
- Need a coordinator to decide which kitchen handles which order (like shard key)

Definition:

Horizontal scaling is the process of **adding more database servers** to handle increased load, instead of upgrading a single server.

In short:

- Vertical = bigger server
- Horizontal = more servers

1️⃣ Why Horizontal Scaling Is Needed

Even with vertical scaling:

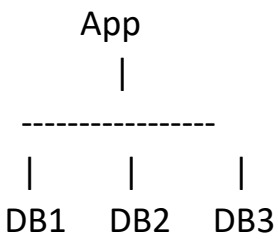
- There's a **hardware limit**
- Single server can become **too expensive**
- Single server = **single point of failure**

Example:

One DB server → 64 CPU, 256 GB RAM
Traffic = 50M users
Still slow → vertical scaling hits its limit

Solution: add more servers → horizontal scaling.

2️⃣ How It Works



- Load is distributed across multiple database servers
- Can separate **reads and writes**
- Can partition **data across servers**

3️⃣ Types of Horizontal Scaling

a) Read Replicas (Most Common First Step)

- Writes go to **primary DB**
- Reads go to **replica DBs**
- Ideal for **read-heavy applications**

App → Primary DB (write)
App → Replica DBs (read)

Example:

- Instagram feed queries → replicas
- Posting new photo → primary

b) Sharding (Advanced Scaling)

- Split data across multiple servers by **shard key**
- Each server handles only part of the data
- Needed when **data grows too large for a single DB**

Shard 1 → users 1–1M
Shard 2 → users 1M–2M
Shard 3 → users 2M–3M

4️⃣ Pros of Horizontal Scaling

✅ Almost unlimited scaling → add more servers
✅ Can separate reads and writes → improves throughput
✅ Reduces load per server → faster queries
✅ Reduces single point of failure (if replicated)

5️⃣ Cons of Horizontal Scaling

❌ More complex architecture
❌ Data consistency can be tricky (replication lag, eventual consistency)
❌ Joins and transactions across shards are harder
❌ Maintenance & monitoring harder

7️⃣ Key Concepts to Know (for Scaling Out)

1. **Replication** → multiple copies of data for reads/failover
2. **Load Balancing** → distribute requests among replicas
3. **Sharding / Partitioning** → split data across servers
4. **Consistency Models** → eventual vs strong consistency

Read/Write Splitting

Definition:

Read/Write Splitting is the practice of **sending write operations to the primary database** and **sending read operations to one or more replicas**.

In short:

- Writes → primary

- Reads → replicas

This **distributes load** and **improves database performance**.

1️⃣ Why Read/Write Splitting Is Needed

In most modern applications:

- **Reads are far more frequent than writes** (often 5–10× more)
- Sending all reads to the primary → bottleneck
- Using replicas for reads → reduces load on primary → better scalability

Challenges

1. **Replication Lag**
    - If you read from a replica immediately after a write → might see **stale data**
    - Solution: read recent writes from **primary**, older data from replicas
2. **Transaction Complexity**
    - Multi-step transactions sometimes require reading from primary to maintain consistency
3. **Load Balancing**

Distribute reads evenly across replicas → prevent one replica from being overloaded

Primary–Replica Replication

Definition:

Primary–Replica Replication is a setup where **one database server handles writes (Primary)** and **one or more replicas handle reads**. Data from the primary is **replicated** to the replicas to keep them up-to-date.

1️⃣ Why Replication is Needed

Even after vertical scaling:

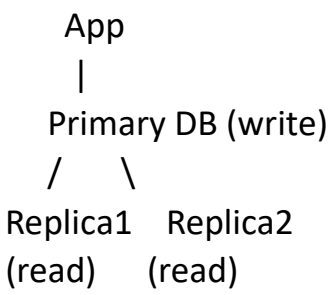
- A single server can handle only so many queries
- **Read-heavy applications** (feeds, dashboards, analytics) put most of the load on **reads**

Replication allows:

- Writes → primary
- Reads → replicas

This **distributes load** and **improves performance**.

2️⃣ Basic Architecture



- App writes → primary
- App reads → replicas
- Replicas **stay in sync** with primary via replication

3️⃣ Types of Replication

a) Synchronous Replication

- Primary waits for replicas to confirm the write before committing
- **Guarantees strong consistency**
- Slower because you wait for all replicas

b) Asynchronous Replication

- Primary commits immediately

- Replicas update **in the background**
- Faster, but replicas may lag (eventual consistency)

4️⃣ Real-World Flow

Example: Social media feed

1. User posts a photo → Primary DB writes the post
2. Replicas receive the new post in background
3. Other users viewing feed → read from replicas
4. User may see the new post after a **small replication delay**

Replication Lag

Definition:

Replication lag is the **delay between when a write happens on the primary database and when that change is visible on the replica**.

In simple terms:

The replica is **slightly behind the primary**.

1️⃣ Why Replication Lag Happens

Even with fast networks:

1. Writes happen on the **primary**
2. Changes are **sent to replicas** via replication log
3. Replicas **apply changes** asynchronously

During this time, **reads from the replica may not see the latest data**.

2️⃣ Types of Lag

a) Network Lag

- Delay in transmitting changes from primary → replica

b) Write/Application Lag

- Replicas process transactions slower than primary

c) Heavy Write Load

- Primary receives many writes → replicas **queue up changes**
- Lag increases

Implications of Replication Lag

- **Eventual Consistency**: replicas might return stale data
- **Read/Write splitting issues**: if an app reads from a replica immediately after a write, it may see old data
- **Analytics dashboards** may show outdated metrics

5️⃣ How to Mitigate Replication Lag

1. **Synchronous Replication**
    - Primary waits for replica confirmation → no lag
    - Slower writes
2. **Semi-Synchronous Replication**
    - Primary waits for **at least one replica**
    - Compromise between speed & consistency
3. **Smart Read Routing**
    - Read from **primary** for recently updated data
    - Read from **replicas** for older data
4. **Monitoring**
    - Track **replication lag** (seconds or transaction offset)
    - Alert if lag exceeds threshold

Database Partitioning

Real-World Analogy

Think of a huge warehouse:

- **One huge room → hard to find items**
- **Divide warehouse into sections by year, category, or region → faster to locate items**

Partitioning = organizing data into logical "sections"

Definition:

Database partitioning is the process of **splitting a large table into smaller, more manageable pieces** called partitions. Each partition contains a subset of the data, often based on a key or range.

Purpose:

- Improves **query performance**
- Reduces **table size for each query**
- Makes **maintenance easier**

1️⃣ Why Partitioning Is Needed

As data grows:

- Table scans become slow
- Indexes grow large → slower searches
- Backups and maintenance take longer

Partitioning solves this by **dividing the table logically**.

Example:

Orders table → 1B rows

Partition by **year**:

- orders_2022 → 200M rows
- orders_2023 → 300M rows
- orders_2024 → 500M rows

Queries on 2024 orders → **scan only that partition** → much faster

2️⃣ Types of Partitioning

a) Range Partitioning

- Split data by ranges of values
- Example: created_at

Partition1 → created_at < '2023-01-01'
Partition2 → '2023-01-01' <= created_at < '2024-01-01'
Partition3 → created_at >= '2024-01-01'

b) List Partitioning

- Split data by **specific list of values**
- Example: country

Partition_US → country='US'
Partition_UK → country='UK'
Partition_IN → country='IN'

---

c) Hash Partitioning

- Split data using **hash function on a column**
- Example: user_id % 4 → 4 partitions
- Good for **evenly distributing data** when ranges are unpredictable

---

d) Composite Partitioning

- Combination of two types, e.g., **range + hash**

- Useful for very large tables with uneven distribution

3️⃣ Benefits of Partitioning

✅ Queries scan **smaller partitions** → faster
✅ Maintenance tasks (backup, archive, delete) are faster
✅ Can combine with **replication & sharding** for massive scale
✅ Reduces **index size per partition**

4️⃣ Drawbacks / Challenges

❌ More complex schema → joins across partitions are harder
❌ Partition key must be chosen carefully → bad key → unbalanced partitions
❌ Some databases have **limited support** for partitioning

Sharding Fundamentals

Real-World Analogy

Think of a massive library:

- **One library = huge, slow, crowded**
- **Shard by author's last name → separate libraries**
- **To find a book → go to correct library**
- **Adding new authors → add new libraries**

Definition:

Sharding is the process of **splitting a database across multiple servers**, where each server (shard) stores only a **subset of the total data**.

Key idea:

- Each shard = **a smaller, independent database**
- Together, all shards = **the complete dataset**

2️⃣ How Sharding Works

a) Shard Key

- Pick a column to split data, e.g., user_id
- Hash or range the key to assign **users to shards**

Example:

Shard1 → user_id 1–1M
Shard2 → user_id 1M–2M
Shard3 → user_id 2M–3M

- Queries for user_id 1.5M → sent to **Shard2 only**
- Reduces load and storage per server

b) Types of Sharding

1. **Horizontal Sharding (Most Common)**
    - Split **rows** across shards
    - Each shard has same schema
    - Example: user_id 1–1M → Shard1
2. **Vertical Sharding**
    - Split **columns** across shards
    - Each shard has part of schema
    - Example: Shard1 → user table, Shard2 → posts table

Horizontal sharding is more common in large-scale web apps.

3️⃣ Benefits of Sharding

✅ Scales writes and reads beyond single server
✅ Each shard handles smaller subset → faster queries

✅ Can add new shards dynamically → almost unlimited scale
✅ Reduces risk of a single shard becoming a bottleneck

🚱 Drawbacks / Challenges

❌ Increased complexity → app must know which shard to query
❌ Cross-shard joins are hard → often avoided
❌ Rebalancing shards (adding/removing servers) is tricky
❌ Backup/restore becomes more complicated

Key Mental Model

- **Partitioning** = splitting a table **inside one database** → like **sections in one warehouse**
- **Sharding** = splitting a table **across multiple databases/servers** → like **multiple warehouses**, each holding part of the inventory

---

Partitioning divides a table into smaller parts within the same database for performance and manageability, whereas sharding splits a table across multiple database servers to scale horizontally for massive datasets.

Consistent Hashing

Real-World Analogy

Think of a pizza delivery zone:

- **4 delivery trucks (servers) → assigned zones on a circular city map**
- **Add a 5th truck → only the nearest slice of the city is reassigned**
- **No need to reshuffle the entire city**

Definition:

Consistent hashing is a technique to **distribute data across multiple nodes (servers/shards) in a way that minimizes data movement when nodes are added or removed**.

In short: it **avoids rehashing the entire dataset** every time your cluster changes.

Consistent hashing is a technique to map data to servers in a distributed system so that when servers are added or removed, only a minimal subset of data needs to move. It's used to scale horizontally without massive data reshuffling.

1️⃣ Why Consistent Hashing Is Needed

Without consistent hashing:

- Suppose you shard data across 4 servers using a simple modulo:

shard = user_id % 4

- Servers: A, B, C, D
- Adding a new server E → all user_id % 5 changes → **most data must move**
- Disaster for large datasets (millions/billions of rows)

Consistent hashing solves this problem.

2️⃣ How It Works

1. **Map both data and servers to a circular hash space** (0–2^{32} for example)
2. **Place data on the next server clockwise from its hash**
3. When a server is added/removed → **only nearby data moves**, rest stays in place

4️⃣ Benefits

✅ Minimal data movement when servers added/removed
✅ Scales horizontally smoothly
✅ Even distribution of data (with virtual nodes)
✅ Widely used in distributed databases (Cassandra, DynamoDB, Redis Cluster)

CAP Theorem

Definition:

The **CAP Theorem**, proposed by Eric Brewer, states that a distributed system can only guarantee **two out of three properties at the same time**:

1. **C — Consistency**
2. **A — Availability**
3. **P — Partition Tolerance**

You can pick **any two**, but not all three simultaneously.

1️⃣ The Three Properties

a) Consistency (C)

- Every read receives the **most recent write**
- Example: If Alice updates her profile picture, everyone sees the new picture immediately

b) Availability (A)

- Every request receives a **response**, even if it's not the latest data
- Example: Even during network issues, system returns some result

c) Partition Tolerance (P)

- System continues to operate **even if network partitions or node failures occur**

- Example: Two data centers can't communicate → system still serves requests

2️⃣ The Trade-offs

System Choice	Explanation
CP (Consistency + Partition Tolerance)	Always consistent even during network issues, but some requests may fail → availability drops
AP (Availability + Partition Tolerance)	Always responds, but data may be slightly stale → eventual consistency
CA (Consistency + Availability)	Perfect reads & writes, but cannot tolerate network partitions → unrealistic in distributed systems

Note: In **real distributed systems**, **Partition Tolerance is mandatory**, because network failures always happen. So the real trade-off is **Consistency vs Availability**.

Analogy:

- Imagine a group chat app with **two servers**:
    - CP → messages are consistent, but some messages might be delayed during network issues
    - AP → messages always delivered, but some users might see old messages temporarily
    -

Why This Matters for Scaling

- In **sharded / replicated systems**, you must decide:
    - Should reads always be consistent (CP) or fast/available (AP)?
- Many modern systems go for **AP with eventual consistency** for scalability (e.g., Instagram feed, caching layers)

CAP Theorem states that a distributed system can only guarantee two out of three properties: Consistency, Availability, and Partition Tolerance. In practice, partition tolerance is mandatory, so distributed systems trade off between consistency and availability.

Caching

Think of a fast-food restaurant:

- **Popular menu items kept ready on counter (cache)**
- **Rare items cooked on demand (DB)**
- **Reduces waiting time for common orders**

Definition:

Caching is the practice of **storing frequently accessed data in a fast-access memory layer** so that future requests can be served quickly without hitting the database.

Caching stores frequently accessed data in fast memory (like Redis or in-app memory) to reduce database load, improve response times, and scale read-heavy applications. Common strategies include cache-aside, write-through, and TTL-based eviction.

1️⃣ Why Caching is Needed

Even with **read replicas** or **sharding**:

- Some queries are very frequent → repeated work
- Databases are slower than memory
- Reducing DB hits improves **latency** and **throughput**

Example:

- User profile read 10,000× per minute

- Without cache → DB handles all 10,000 queries
- With cache → DB hit only once, next requests served from memory

2️⃣ Types of Caches

a) In-Memory Cache

- Data stored **in application memory**
- Fastest (nanoseconds–microseconds)
- Limited by application memory size

Example: Node.js using Map or LruCache

```
const cache = new Map();
cache.set("user:1", userData);
const data = cache.get("user:1"); // O(1) access
```

b) Distributed Cache (e.g., Redis, Memcached)

- Separate **fast key-value store** shared by multiple servers
- Scales horizontally → multiple app servers can use same cache
- Data persists for **TTL (Time To Live)** or eviction

```
GET user:1
SET user:1 {data} EX 60 // expire in 60 seconds
```

3️⃣ Cache Strategies

Strategy	Description
Cache Aside (Lazy Loading)	Load data from DB into cache **on demand**
Write-Through	Write to DB **and cache simultaneously**
Write-Back (Write-Behind)	Write to cache first, DB updated later

Strategy	Description
Read-Through	Application reads from cache, cache loads DB if missing

4️⃣ Cache Eviction Policies

- **LRU (Least Recently Used)** → evict least used data first
- **LFU (Least Frequently Used)** → evict least popular data
- **TTL (Time To Live)** → auto-expire data after set time

5️⃣ Benefits

✅ Reduces database load
✅ Improves response times dramatically
✅ Scales read-heavy applications
✅ Can work across multiple app servers

6️⃣ Drawbacks / Challenges

❌ Stale data → cache may not reflect latest DB writes
❌ Complexity in cache invalidation
❌ Memory is limited → not all data can fit in cache
❌ Consistency issues in distributed systems

Cache Invalidation Strategies

Definition:

Cache invalidation is the process of **updating or removing stale data from the cache** when the underlying database changes, ensuring clients get correct data.

“Stale cache = bugs waiting to happen.”

Cache invalidation ensures cached data stays consistent with the database. Strategies include TTL/expiration, write-through, write-behind, cache-aside (lazy loading), and explicit invalidation.

Real-World Analogy

Think of a **restaurant menu board**:

- TTL → Menu board updated every hour
- Write-through → Menu updated immediately as dishes change
- Write-behind → Menu updated later asynchronously
- Cache-aside → Check kitchen if dish exists, then update board
- Explicit → Chef manually updates board after changing dish

1️⃣ Why Invalidation Matters

- Writes happen in the DB → cache may still have old data
- If app reads from stale cache → **data inconsistency**
- Especially critical in **financial apps, e-commerce carts, social feeds**

2️⃣ Main Strategies

a) Time-to-Live (TTL) / Expiration

- Each cache entry has a **set lifetime**
- After TTL → cache auto-expires → next read fetches DB
- Pros: simple, automatic
- Cons: stale data until expiry, hard to pick perfect TTL

```
SET user:1 {data} EX 60  // expires in 60s
```

b) Write-Through Cache

- Every **write/update** goes to **cache and DB simultaneously**

- Ensures cache is always **up-to-date**
- Pros: strong consistency
- Cons: slower writes

```
db.write(user)
cache.write(user)
```

c) Write-Behind / Write-Back Cache

- Write goes to **cache first**, DB updated asynchronously
- Pros: very fast writes
- Cons: risk of data loss if cache crashes, eventual consistency

```
cache.write(user)
async → db.write(user)
```

d) Cache-Aside (Lazy Loading)

- App checks **cache first**
- If miss → load from DB, populate cache
- On update → invalidate cache entry manually
- Most commonly used in web apps

```
data = cache.get(key)
if (!data) {
   data = db.get(key)
   cache.set(key, data)
}
```

e) Explicit Invalidation

- App **manually removes** or updates cache after DB change
- Often combined with cache-aside

```
db.update(user)
cache.delete(user:1)
```